Detailed Example of Bin-Based Removal of 'No' Rows

HR Employee Attrition Dataset

This document explains exactly how the given code decides which 'No' rows to remove. The complete bin is never deleted. The bins are temporary and are used to calculate a Removal_Score for individual 'No' employees.

1. Starting with a small example

To understand the real dataset, imagine a small dataset containing 13 employees and one feature: Age.

Employee	Age	Attrition
A	22	Yes
B	25	No
C	27	No
D	28	No
E	32	Yes
F	34	No
G	35	No
H	37	No
I	42	Yes
J	44	No
K	46	No
L	48	No
M	49	No

In this example: Yes = 3 and No = 10.

2. Create temporary Age bins

For easy understanding, divide Age into three temporary bins.

Bin	Age Range
Bin 1	20–30
Bin 2	30–40
Bin 3	40–50

3. Count Yes and No in each bin

The employees are temporarily grouped according to Age.

Age Bin	Yes	No	Maximum No (2×Yes)	Excess No
20–30	1	3	2	1
30–40	1	3	2	1
40–50	1	4	2	2

For the 40–50 bin, there is 1 Yes and 4 No. The maximum allowed No is 1×2=2. Therefore, 4−2=2 No employees are considered excess in that bin.

4. Calculate Excess Ratio

Excess Ratio = Excess No ÷ Total No in the bin.

Bin	Calculation	Excess Ratio

20–30	1 ÷ 3	0.333
30–40	1 ÷ 3	0.333
40–50	2 ÷ 4	0.500

The 40–50 bin has the highest Age-based overrepresentation score. However, this does NOT mean that the whole 40–50 bin will be removed.

5. The score is given to individual No rows

The code finds only the No employees inside each bin and adds that bin's score to their Removal_Score.

Employee	Age	Attrition	Age Removal Score
B	25	No	0.333
C	27	No	0.333
D	28	No	0.333
F	34	No	0.333
G	35	No	0.333
H	37	No	0.333
J	44	No	0.500
K	46	No	0.500
L	48	No	0.500
M	49	No	0.500

J, K, L and M all receive 0.500 for Age. The code does not choose one employee from this bin based only on Age.

6. Scores from all eight features are combined

The actual code repeats the process for Age, MonthlyIncome, TotalWorkingYears, YearsAtCompany, YearsInCurrentRole, YearsWithCurrManager, DistanceFromHome and JobLevel. A No employee can therefore receive a score from several features. The scores are added to create one final Removal_Score.

Employee	Age	Income	Working Years	JobLevel	Total
A	0.50	0.00	0.40	0.60	1.50
B	0.00	0.30	0.00	0.20	0.50
C	0.40	0.50	0.30	0.00	1.20
D	0.00	0.00	0.10	0.00	0.10

These numbers are illustrative. They demonstrate the scoring mechanism and are not the actual scores of specific employees in the real dataset.

7. Rank individual No employees

After all feature scores are added, the code sorts the No employees from highest Removal_Score to lowest.

Rank	Employee	Total Removal Score
1	A	1.50
2	C	1.20
3	B	0.50
4	D	0.10

8. Remove only the required number

Suppose we need to remove 3 No rows. The first three individual No rows in the sorted list are removed.

Employee	Score	Decision

A	1.50	REMOVE
C	1.20	REMOVE
B	0.50	REMOVE
D	0.10	KEEP

Therefore, the code does not remove a complete bin. It removes individual row indices until the exact required number is reached.

9. Applying the logic to the actual dataset

The actual dataset starts with 1233 No and 237 Yes. For the desired 2:1 ratio:

Target No = 237 × 2 = 474

Rows to remove = 1233 − 474 = 759

Class	Before	After
No	1233	474
Yes	237	237
Total	1470	711

The program calculates a Removal_Score for the No employees, sorts them, and selects exactly 759 individual No rows. The 237 Yes rows remain.

10. What happens to a high-scoring bin?

Suppose the 40–50 bin has 1 Yes and 4 No. Its high Excess_Ratio gives its No rows a higher Age score. But if only two of those employees are among the final highest-scoring rows, only those two are removed.

Before: 40–50 → Yes = 1, No = 4

After selecting two No rows: 40–50 → Yes = 1, No = 2

The other two No employees in the same bin remain. Therefore, the bin itself was never deleted.

11. Original values remain unchanged

The bin columns are temporary. The final dataset is made using the original dataframe:

```
df_modified = df.drop(index=rows_to_remove).copy()
```

If a kept employee originally had Age=44, MonthlyIncome=5993 and TotalWorkingYears=8, those exact values remain. The final CSV does not replace them with bin labels.

12. Complete process

1. Original HR dataset
2. Create temporary bins
3. Count Yes and No inside every bin
4. Calculate maximum allowed No using the 2:1 rule
5. Calculate Excess and Excess_Ratio
6. Give the score to individual No rows in that bin
7. Repeat for all eight features
8. Add the scores for each No employee
9. Sort individual No employees by Removal_Score
10. Remove exactly the required number of No rows
11. Keep original values of remaining rows
12. Save the final balanced dataset

13. Viva explanation

We temporarily divide numerical features into bins to identify where the No class is overrepresented. Each No employee receives a score based on the overrepresentation of the bins they belong to across multiple features. We

then remove only the required number of highest-scoring individual No rows, not the entire bin, while preserving the original values of the remaining employees.